Introduction to ROS: Autonomous Driving Group Project

Dean Mercer, Lukas Evers, Mena Kuzman, Sammy Breen

1 Introduction

This is the documentation for the autonomous driving group project of our team. Section 2 gives an overview of the architecture, the used ROS packages and nodes, and the TF frames used. Section 3 explains the details about the implementation in more detail. Finally, section 4 explains our division of tasks.

2 Overview

The driving software is structured into three major components: perception, navigation, and control. These are implemented across different packages and nodes as shown in the ROS graph in figure 1.

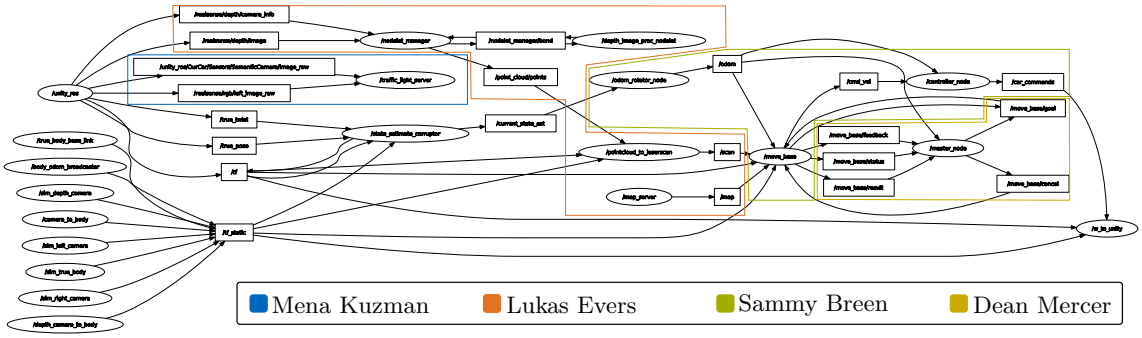

Figure 1: ROS graph. Participation of the team members is indicated by colored boxes.

The perception pipeline consists of two parts, the obstacle detection and the traffic light detection. The depth image sensor message is processed by the *depth_image_proc* package to provide a pointcloud. A global occupancy map was created beforehand with said pointcloud by using the *octomap_server* package in order to map static obstacles. During operation of the vehicle the incoming pointcloud is converted into a 2D laserscan within a clipped range of the pointcloud height. The laserscan is used as source for the creation of the local costmap functioning as the basis for the local planning done by the *move_base* package, described below. The traffic light detection is implemented as a ROS service, which is used by the car to check and obey the traffic light signals. The service sends the traffic light status with a custom message type called *TrafficLight*.

Central to our driving software is the ROS package *move_base*, which provides an interface to run and configure global and local planners for the navigation of the vehicle [1]. *move_base* uses a global and local costmap for the global and local planners respectively. Figure 3 shows the static map of the road network, which is provided by a *map_server* node and used in the global costmap [2]. The local costmap is created from the laser scan data published by *pointcloud_to_laserscan*. The *NavFnROS* planner from the *navfn* package is used as the global planner, which creates a path from the vehicles current position to a goal pose with the Dijkstra algorithm [3]. An optimization-based local planner called *teb_local_planner* finally creates the local trajectory, which then is published as desired velocities and yaw rates in a twist message [4].

The *master_node* node is responsible for publishing the next subgoal in order to follow the predefined track through the road network. The subgoals are defined as poses in a text file. The master node interfaces with the traffic light detection service to control stopping at red lights.

The desired velocity and yaw rate are used to control the acceleration and steering angle inputs of the vehicles actuators in the *controller_pkg* and the *controller_node*. The resulting control outputs are sent to the Unity simulation as actuator messages. The details of the controller implementation are explained in section 3.3.

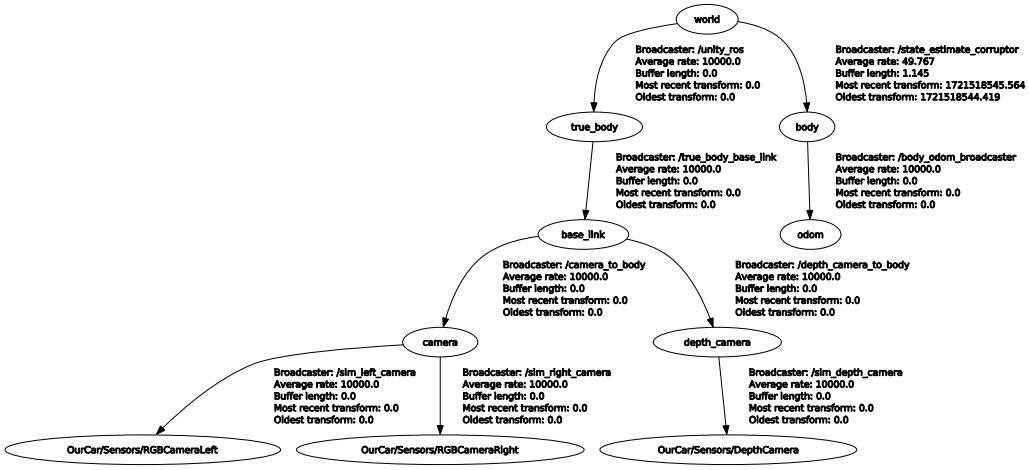

Figure 2: TF frames

Figure 2 shows the TF tree employed for the autonomous vehicle. The Unity simulation provides the world frame from which two other frames are created. The *true_body* frame represents the local frame of the vehicle. The body frame is created by the *state_estimate_corruptor_node*. In both of these frames the y-axis is aligned with the longitudinal dimension of the vehicle. However, for the navigation, coordinate frames whose x-axis is aligned with the vehicle are required. For this purpose the two frames *base_link* and *odom* are utilized, which are created by turining the *true_body* and the *body* frames by 90 degrees. The odometry message published by the *state_estimate_corruptor_node* is turned into the *odom* frame with the help of a simple node called *odom_rotator_node*. Additional frames for the camera sensors are created with static transformations.

Table 1 summarizes the external ROS packages that are used in this project.

Table 1: External code used in the project

Name	**Description**	**Ref.**
depth_image_proc	Converts the depth image to a point cloud	[5]
octomap_server	Used to map the environment	[6]
pointcloud_to_laserscan	Converts a Pointcloud into a 2D Laserscan	[7]
map_server	Used to load and publish a saved occupancy map of the environment	[2]
move_base	Provides a ROS interface for configuring, running, and interacting with the navigation stack	[1]
navfn	Global planner that uses the Dijkstra algorithm	[3]
teb_local_planner	Optimization based local planner that plans the trajectory	[4]
cv_bridge	Acts as a bridge between ROS image message and OpenCV	[8]
image_transport	For image transportation and compression	[9]

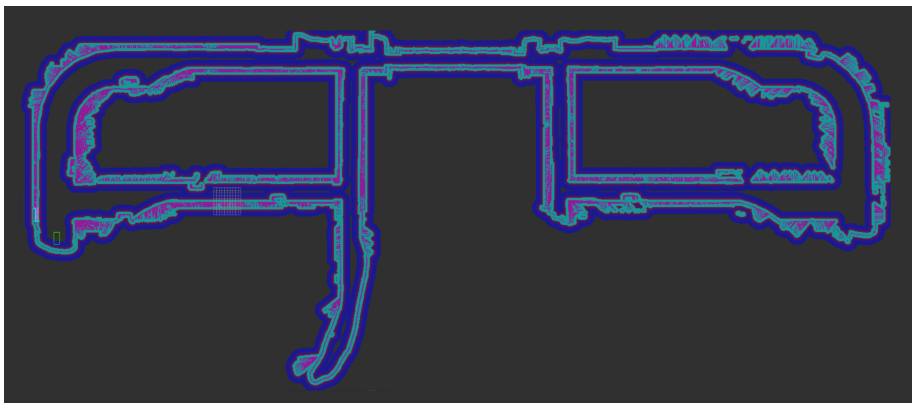

Figure 3: Map of the road network. The map was created with *octomap_server* [6]. Visualization in RViz.

3 Implementation

In this section we will explain the implementation details for the different components of our software stack.

3.1 Perception

3.1.1 Obstacle detection

One prerequiste for succesful autonomous navigation are suitable sensor sources from which an enriched environment representation can be built. An Occupancy Grid Map was recorded with the help of the octomap_server package. Parallel to the octree maps a 2D occupancy map can be saved via a service call. At the start of the system this map is loaded with the map_server package. This map contains all static obstacles present during the mapping phase and the map is used to generate a global costmap as a basis for the global planner. To generate octomaps, the depth image from the stereo camera is converted into a pointcloud via the depth_image_proc package. In a next processing step this 3D pointcloud is condensed into a 2D laserscan message which is used by the local planner to populate the local costmap with static and dynamic obstacles. This is accomplished by the pointcloud_to_laserscan package. The usable points from the pointcloud are clipped with an lower and upper bound to not detect the ground as obstacle and to discard obstacles higher than the car height plus some margin.

3.1.2 Traffic light detection

A ROS service is created to detect the traffic lights and their colors. This service obtains data from the semantic and rgb cameras, and converts this data into a *TrafficLight* message with the status of an observed traffic light. Clients can then subscribe to the service and obtain information about the traffic lights. The images received from the semantic and rgb cameras are cropped to only include the central region of the image, which serves the task of avoiding the detection of multiple traffic lights that was encountered during the testing phase. The *cv_bridge* is used to enable using ROS image message in conjunction with OpenCV. which is used for image processing, in addition to the *image_transport* which is used to transport the image and abstract its complexity. [8, 9].

3.2 Navigation

The navigation package is responsible for creating a collision free trajectory that follows the predefined track from start to finish while obeying traffic lights. To achieve this task we are using a global planner that plans a path to a specified goal pose, a local planner that plans a collision free trajectory along the global path and a waypoint publisher that

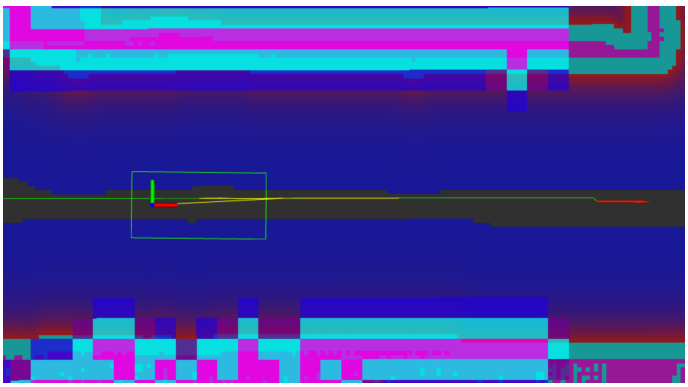

Figure 4: Planner scene. The vehicle, shown as a green outline, drives along the global plan towards a goal pose. The plot also shows the global and local costmap which are used to plan collision-free trajectories.

publishes subgoals along the track. Figure 4 shows an example scene of the navigation, where the vehicle drives towards a goal pose.

3.2.1 Global Planning

The global planner is responsible for finding a path from the current position to a goal across the road network. This is done with the Dijkstra algorithm implemented in the ROS package *navfn* [3]. This planner needs a global costmap which is created from a static map and specifies the cost of each cell on a 2D voxel grid. The cost of each cell is determined by the static map. With an inflation layer the cost of cells adjacent to occupied cells is manipulated such that global plans further away from them are preferred.

3.2.2 Local Planning

While the global planner only provides a path to the goal, the local planner is required to plan the trajectory without colliding with any obstacles. Similarly to the global planner, the local planner uses a local costmap represented by a 2D voxel grid. The local costmap is created by *move_base* with the laser scan output provided by the perception pipeline. We are using the *teb_local_planner* for this task, which is an optimization-based planner and supports non-holonomic vehicle dynamics. With the local plan the desired velocity and yaw rate are calculated and published on the */cmd_vel* topic.

One issue we have encountered with this planner was the high computational effort of the optimization process. Due to this we limited the frequency of the planner to 10 Hz and used a relatively coarse grid for the local costmap with a resolution of 0.5 m in order to improve the performance. Depending on the hardware the planner still sometimes fails to achieve the specified frequency due to longer planning cycle execution times, which results in a warning by *move_base*.

3.2.3 Goal Publishing

The *master_node* node loads a list of waypoints from a text file on startup and subsides to the optometry messages. Each time the ego car comes within range of the target waypoint, the next one is loaded and sent to *move_base* as the next goal. The node contains a client to the traffic light detection service to control the state of the vehicle. If an observed traffic light is red, the node stops publishing goals, which causes the vehicle to stop. The goal publishing resumes, when the traffic light is green.

3.3 Control

The inputs of the the vehicle are the acceleration a, steering angle δ and braking command. The local planner of *move_base* publishes a desired velocity v_d and a desired yaw rate $\dot{\psi}_d$.

Therefore, one controller that handles the acceleration and braking commands and one that controls the steering angle are required. The controllers are implemented in the *controller_pkg* package and the *controller_node* node.

3.3.1 Acceleration control

At each iteration of the control loop the current longitudinal velocity v_c is derived from the state estimation. The acceleration is then controlled with a simple PID controller:

$$u_a(t) = K_P e(t) + K_I \int_0^t e(t)\,dt + K_D \dot{e}(t) \tag{1}$$

with $e(t) = v_d - v_c$.

3.3.2 Steering angle control

Using a kinematic single track model the relationship between the yaw rate and the steering angle can be easily determined:

$$\dot{\psi} = \frac{v \tan \delta}{L} \tag{2}$$

with the velocity of the vehicle v and the wheelbase L. With this relationship the desired and the current turning angle, δ_d and δ_c respectively, can be easily calculated. Afterwards a P controller is employed to control the steering angle:

$$u_\delta(t) = K_\delta(\delta_d - \delta_c) \tag{3}$$

4 Division of tasks

This section explains who worked on which part. Table 2 gives an overview of the sub tasks and who worked on it. The alternative depth map converter Dean Mercer worked on was not used in the final project. Substantial efforts went into the parameterization of move_base, which was done jointly by Sammy Breen, Lukas Evers and Dean Mercer.

Table 2: Division of tasks

Task	**Responsible Member(s)**
Perception Pipeline: Depth image to laserscan	Lukas Evers
Alternative depth map converter	Dean Mercer
Mapping: Creation of static map	Lukas Evers
Traffic light detection service	Mena Kuzman
Navigation setup including global and local planning	Sammy Breen
Waypoint publishing	Dean Mercer
Integration of traffic light detection into navigation	Mena Kuzman
Controller implementation	Sammy Breen
Paramter Tuning	Sammy Breen, Lukas Evers, Dean Mercer

5 Installation and Usage

For installation and usage please refer to the README file in the repository.

References

[1] "move_base - ROS Wiki." https://wiki.ros.org/move_base, 2024.

[2] "map_server - ROS Wiki." https://wiki.ros.org/map_server, 2024.

[3] "navfn - ROS Wiki." https://wiki.ros.org/navfn, 2024.

[4] "teb_local_planner - ROS Wiki." https://wiki.ros.org/teb_local_planner, 2024.

[5] "depth_image_proc - ROS Wiki." https://wiki.ros.org/depth_image_proc, 2024.

[6] "octomap_server - ROS Wiki." https://wiki.ros.org/octomap_server, 2024.

[7] "pointcloud_to_laserscan - ROS Wiki." https://wiki.ros.org/pointcloud_to_laserscan, 2024.

[8] "cv_bridge - ROS Wiki." https://http://wiki.ros.org/cv_bridge, 2024.

[9] "image_transport - ROS Wiki." https://http://wiki.ros.org/image_transport, 2024.